



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, <u>Subset Sum</u> , Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
 - Matrix chain Multiplication
 - Assembly line Scheduling
 - Longest Common Subsequence
 - 0/1 Knapsack Problem
 - TSP- Travelling Salesman Problem
- **Backtracking**
 - N-queens Problem
 - **Sum of Subsets**
 - Graph Coloring
- **Branch and Bound**
 - LIFO and FIFO Methods
 - Job Selection Problem
 - 0/1 Knapsack Problem



CBS3003- Design and Analysis of Algorithms- References

Text Books:

1. Horowitz, E., Sahni, S., & Rajasekaran, S. Fundamental of computer algorithms, Hyderabad, Universities Press; Second edition, 2008.
2. Kleinberg J, Tardos E. Algorithm design. Pearson Education India; 2006.

Reference Books:

1. Knuth Donald E, “ Art of Computer Programming: Fundamental algorithms”, Third edition, Pearson Publishers, 2011.
2. Pat Morin, “ Open Data Structures: An Introduction(Open Paths to Enriched Learning)”, 31 st Edition, UBC Press, 2013.



BACKTRACKING

- One of the mostly used general design technique which follows a strategy for guessing at a solution and backing up when deadlock situation is reached.
- Many problems which deal with **searching for a set of solutions** or **which ask for an optimal solution satisfying some constraints** can be solved using the backtracking.
- In many applications, the desired solution is expressible as an n-tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i .
- Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$



BACKTRACKING

- **Explicit and Implicit Constraints**

- Explicit constraints are rules that restrict each X_i to take on values only from a given set.
- Explicit constraints depend on the particular instance “ I ” of the problem being solved. All tuples that satisfy the explicit constraints define a possible solution space for “ I ”.
- Implicit constraints are rules that determine which of the tuples in the solution space of “ I ” satisfy the criterion function.



Sum of Subsets - Idea

- Given a set of n distinct positive integers, find the combination of numbers that sum to given value M .
- Sum of subsets problem is analogous to the Knapsack Problem. Knapsack Problem tries to fill the knapsack using a given set of items to maximize the profit.
- Items are selected in such a way that the total weight in the knapsack does not exceed the capacity of the knapsack.
- The inequality condition in the knapsack problem is replaced by equality in the sum of subsets problem.
- Given the set of n positive integers, $W = \{w_1, w_2, \dots, w_n\}$, and given a positive integer M , the sum of the subset problem can be formulated as follows (where w_i and M correspond to item weights and knapsack capacity in the knapsack problem):

$$\sum_{i=1}^n w_i x_i = M$$

Where, $x_i \in \{0, 1\}$

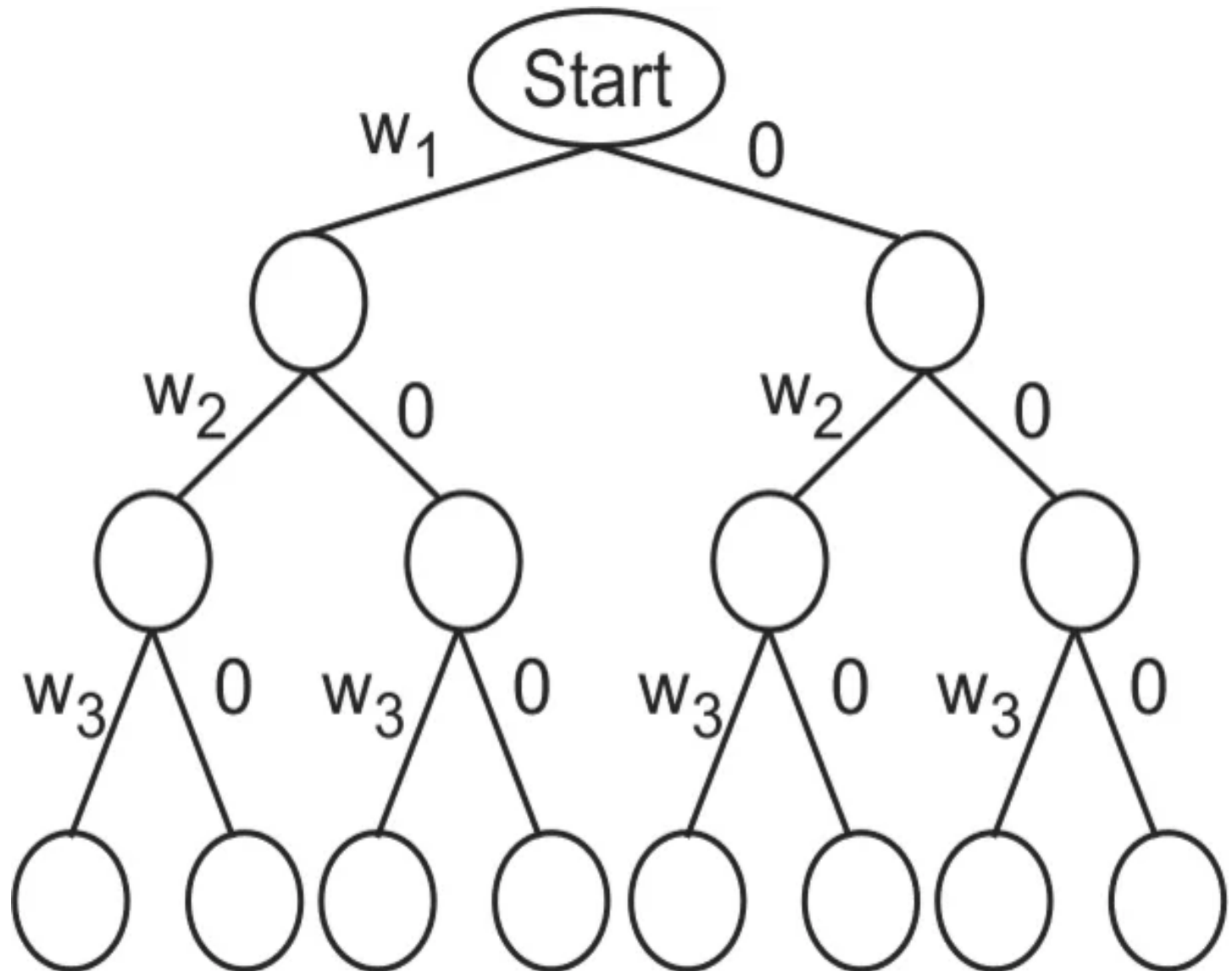


Sum of Subsets

- Numbers are sorted in ascending order, such that $w_1 < w_2 < w_3 < \dots < w_n$.
- The solution is often represented using the solution vector X . If the i^{th} item is included, set x_i to 1 else set it to 0.
- In each iteration, one item is tested. **If the inclusion of an item does not violate the constraint of the problem, add it.** Otherwise, **backtrack, remove the previously added item, and continue the same procedure for all remaining items.**
- The solution is easily described by the state space tree.
- Each **left edge denotes the inclusion of w_i** and the **right edge denotes the exclusion of w_i .**
- Any path from the **root to the leaf forms a subset.**



Sum of Subsets – State space tree for $n=3$



Sum of Subsets

- In Backtracking algorithm as we go down along depth of tree we **add** elements so far, and if the added sum is satisfying explicit constraints, we will continue to generate child nodes further.
- Whenever the constraints are not met, we **stop** further generation of sub-trees of that node, and **backtrack** to previous node to explore the nodes not yet explored.
- We need to explore the nodes along the **breadth** and **depth** of the tree.
- Generating nodes along **breadth** is controlled by loop and nodes along the **depth** are generated using **recursion** (post order traversal).



Sum of Subsets- Algorithm

1. Start with an **empty set**.
2. **Add** the next element from the list to the set.
3. If the subset is equal to sum M , then stop with that subset as solution.
4. If the subset is **not feasible** or if we have reached the **end of the set**, then **backtrack** through the subset until we find the most suitable value.
5. If the subset is **feasible (sum of subset $< M$)** then go to step 2.
6. If we have visited all the elements without finding a suitable subset and **if no backtracking is possible then stop** without solution.



Sum of Subsets- Problem

- Consider the sum-of-subset problem, $n = 4$, $\text{Sum} = 13$, and $w_1 = 3$, $w_2 = 4$, $w_3 = 5$ and $w_4 = 6$. Find a solution to the problem using backtracking. Show the state-space tree leading to the solution. Also, number the nodes in the tree in the order of recursion calls.
- **Solution:**
- The correct combination to get the sum of $M = 13$ for given $W = \{3, 4, 5, 6\}$ is $[3, 4, 6]$.
- The solution vector for $[3, 4, 6]$ would be $X = [1, 1, 0, 1]$ because 3rd element 5 is not chosen, so $X[3] = 0$.
- Let's derive the solution using Backtracking. The numbers in W are already sorted.
- Set $X = [0, 0, 0, 0]$; Set $\text{Sum} = 0$. Sum indicates summation of selected numbers from W .

Sum of Subsets- Problem

Step 1 : $i = 1$, Adding item w_i

$$\text{Sum} = \text{Sum} + w_i = \text{Sum} + w_1 = 0 + 3 = 3$$

$\text{Sum} \leq M(13)$, so add item i to solution set.

$$X[i] = X[1] = 1 \Rightarrow X = [1, 0, 0, 0]$$

Step 2 : $i = 2$, Adding item w_2

$$\text{Sum} = \text{Sum} + w_i = \text{Sum} + w_2 = 3 + 4 = 7$$

$\text{Sum} \leq M(13)$, so add item i to solution set.

$$X[i] = X[2] = 1 \Rightarrow X = [1, 1, 0, 0]$$



Sum of Subsets- Problem

Step 3 : $i = 3$, Adding item w_3

$$\text{Sum} = \text{Sum} + w_i = \text{Sum} + w_3 = 7 + 5 = 12$$

$\text{Sum} \leq M(13)$, so add item i to solution set.

$$X[i] = X[3] = 1 \Rightarrow X = [1, 1, 1, 0]$$

Step 4 : $i = 4$, Adding item w_4

$$\text{Sum} = \text{Sum} + w_i = \text{Sum} + w_4 = 12 + 6 = 18$$

$\text{Sum} > M(13)$, so backtrack and remove the previously added item from the solution set.

$$X[i] = X[3] = 0 \Rightarrow X = [1, 1, 0, 0].$$

Update Sum accordingly. So, $\text{Sum} = \text{Sum} - w_3 = 12 - 5 = 7$ And don't increment i



Sum of Subsets- Problem

Step 5 : $i = 4$, Adding item w_4

$$\text{Sum} = \text{Sum} + w_i = \text{Sum} + w_4 = 7 + 6 = 13$$

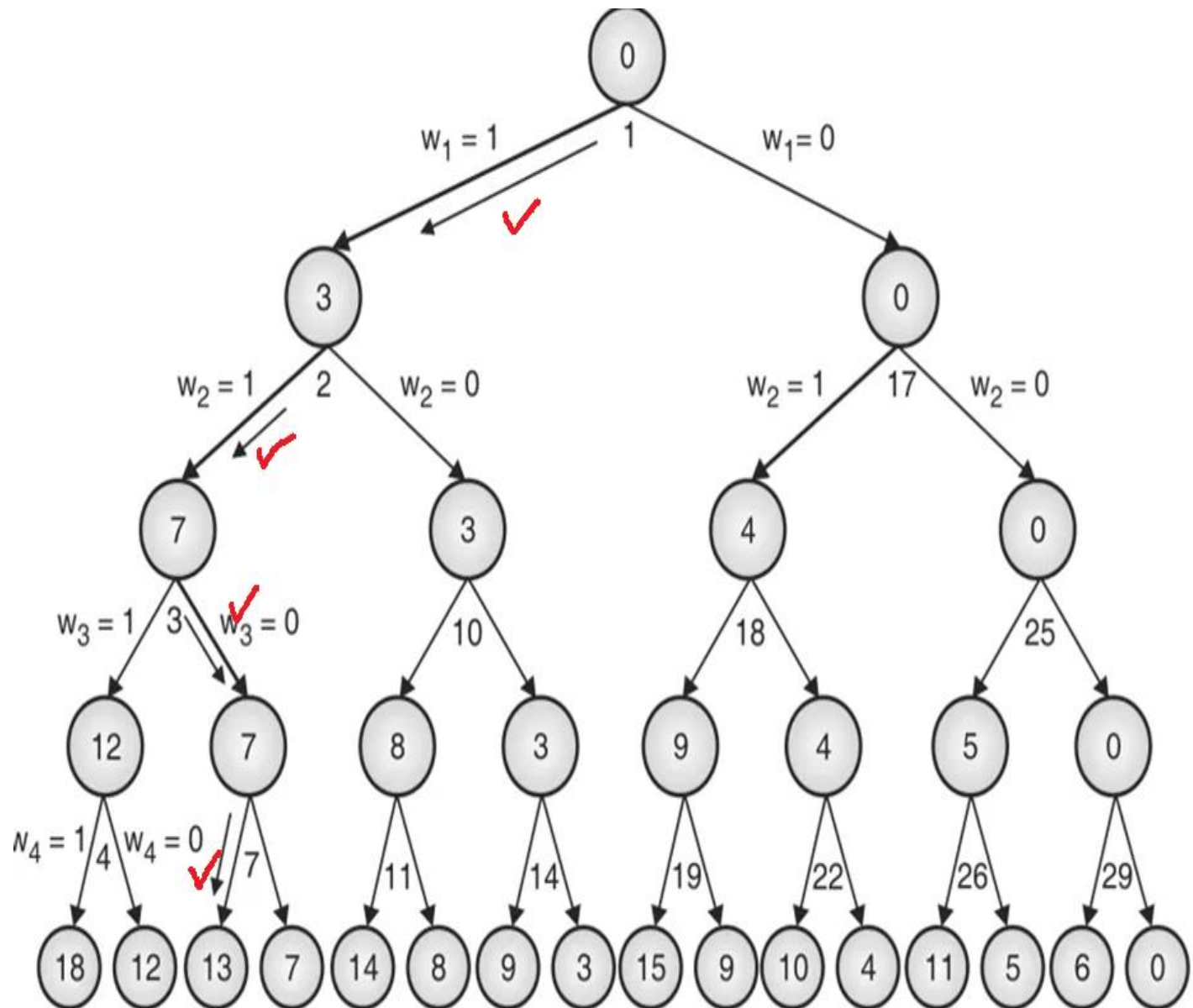
$\text{Sum} = M(13)$, so **solution is found** and add item i to solution set.

$$X[i] = X[4] = 1 \Rightarrow X = [1, 1, 0, 1]$$

A complete state space tree for given data is shown in Fig



Sum of Subsets- State Space Tree



Sum of Subsets- Algorithm

Algorithm SumOfSub(s, k, r)

// Find all subsets of $w[1 : n]$ that sum to m . The values of $x[j]$,

// $1 \leq j < k$, have already been determined. $s = \sum_{j=1}^{k-1} w[j] * x[j]$

// and $r = \sum_{j=k}^n w[j]$. The $w[j]$'s are in nondecreasing order.

// It is assumed that $w[1] \leq m$ and $\sum_{i=1}^n w[i] \geq m$.

{

// Generate left child. Note: $s + w[k] \leq m$ since B_{k-1} is true.

$x[k] := 1$;

if ($s + w[k] = m$) **then write** ($x[1 : k]$); // Subset found

// There is no recursive call here as $w[j] > 0$, $1 \leq j \leq n$.

else if ($s + w[k] + w[k + 1] \leq m$)

then SumOfSub($s + w[k], k + 1, r - w[k]$);

// Generate right child and evaluate B_k .

if ($(s + r - w[k] \geq m)$ **and** ($s + w[k + 1] \leq m$)) **then**

{

$x[k] := 0$;

SumOfSub($s, k + 1, r - w[k]$);

}

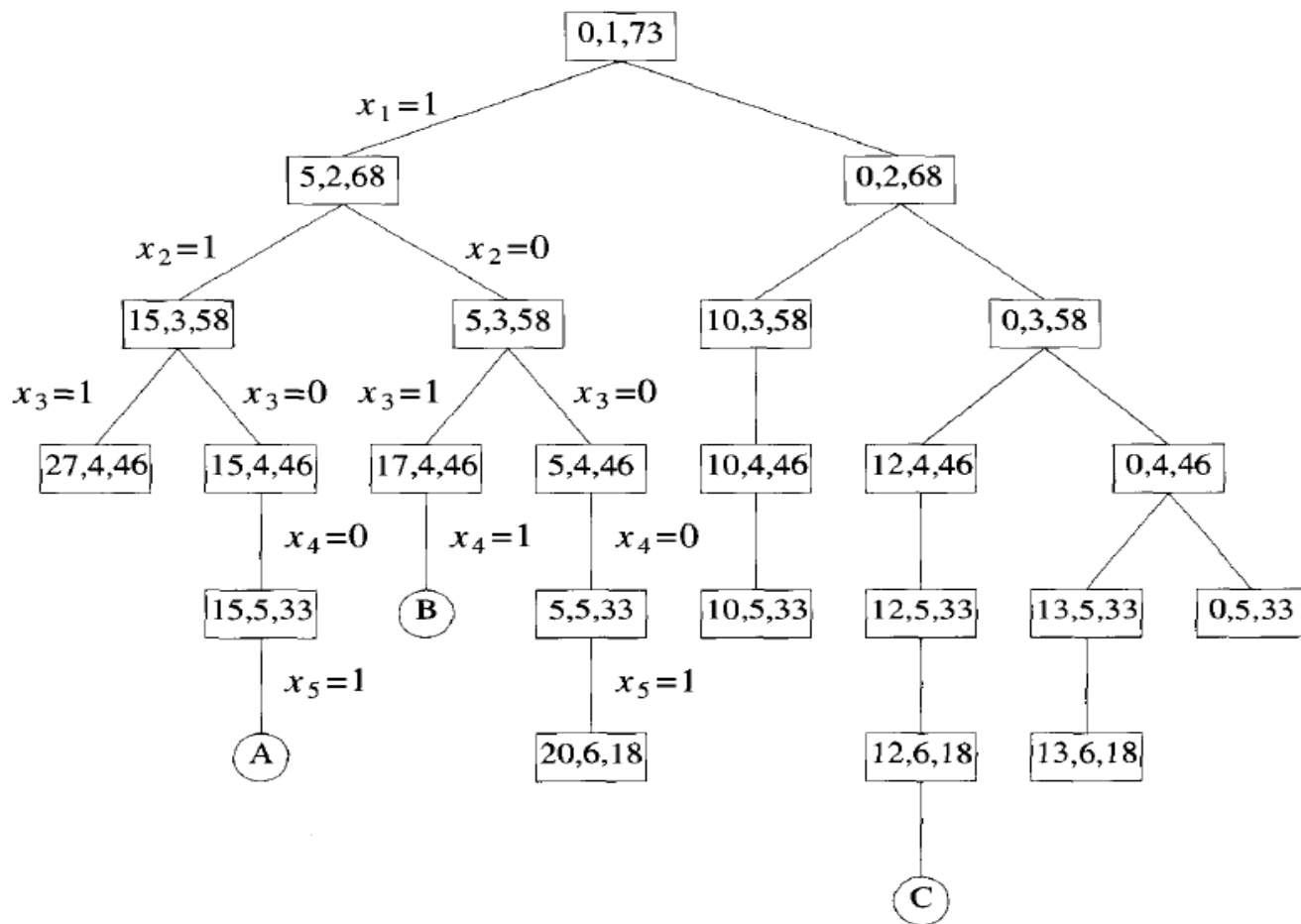
}



Example 2:

I. Solve sum of subsets problem on data :

$M = 30$ and $w = \{5, 10, 12, 13, 15, 18\}$



Sum of Subsets- Time Complexity Analysis

- It is intuitive to derive the complexity of sum of the subset problem. In the state-space tree, at level i , the tree has 2^i nodes. So, given n items, the total number of nodes in the tree would be $1 + 2 + 2^2 + 2^3 + \dots + 2^n$.
- $$\begin{aligned} T(n) &= 1 + 2 + 2^2 + 2^3 + \dots + 2^n = 2^{n+1} - 1 \\ &= O(2^n) \end{aligned}$$



Sum of Subsets- Try yourself

1. Solve sum of subsets problem on the following data: $W = (w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$ and $M = 31$.

